Python Lab Assignment

Name: **Shubh Routh**

Student ID: **0627431**

In [54]:
```
##### Imports

import pandas as pd
import numpy as np
```

In [55]:
```
#### Data Loading & Understanding

### TASK 1:- Load both datasets and display the first 5 rows.

## DATASET 1

df1 = pd.read_csv("German-Credit_1.csv")

print("First 5 Rows of Dataset 1")
display(df1.head())
```

```
First 5 Rows of Dataset 1
```

	OBS	DURATION	NEW_CAR	USED_CAR	FURNITURE	RADIO_TV	EDUCATION	RE
0	1	6.0	0.0	0.0	0.0	1.0	0.0	
1	2	48.0	0.0	0.0	0.0	1.0	0.0	
2	3	12.0	0.0	0.0	0.0	0.0	1.0	
3	4	42.0	0.0	0.0	1.0	0.0	0.0	
4	5	24.0	1.0	0.0	0.0	0.0	0.0	

5 rows × 23 columns

In [56]:
```
## DATASET 2

df2 = pd.read_csv("German-Credit_2.csv")

print("First 5 Rows of Dataset 2")
display(df2.head())
```

```
First 5 Rows of Dataset 2
```

	OBS	CHK_ACCT	HISTORY	SAV_ACCT	EMPLOYMENT	PRESENT_RESIDENT	JOB
0	1	0	4	4	4	4	2
1	2	1	2	0	2	2	2
2	3	3	4	0	3	3	1
3	4	0	2	0	3	4	2
4	5	0	3	0	2	4	2

In [57]:
```
### TASK 2:- Check the shape of both datasets

print("Shape of Dataset 1:", df1.shape)
print("Shape of Dataset 2:", df2.shape)
```

```
Shape of Dataset 1: (1000, 23)
Shape of Dataset 2: (1000, 7)
```

In [58]:
```
### TASK 3:- Display column names and identify numerical features.

## COLUMN NAMES

# DATASET 1

print("\nColumns in Dataset 1")
print(df1.columns.tolist())
```

```
Columns in Dataset 1
['OBS', 'DURATION', 'NEW_CAR', 'USED_CAR', 'FURNITURE', 'RADIO_TV', 'EDUCATIO
N', 'RETRAINING', 'AMOUNT', 'INSTALL_RATE', 'CO_APPLICANT', 'GUARANTOR', 'REA
L_ESTATE', 'PROP_UNKN_NONE', 'AGE', 'OTHER_INSTALL', 'RENT', 'OWN_RES', 'NUM_CR
EDITS', 'NUM_DEPENDENTS', 'TELEPHONE', 'FOREIGN', 'RESPONSE']
```

In [59]:
```
# DATASET 2

print("\nColumns in Dataset 2")
print(df2.columns.tolist())
```

```
Columns in Dataset 2
['OBS', 'CHK_ACCT', 'HISTORY', 'SAV_ACCT', 'EMPLOYMENT', 'PRESENT_RESIDENT', 'J
OB']
```

In [60]:
```
## NUMERICAL FEATURES

# DATASET 1

print("\nNumerical Features in Dataset 1")
print(df1.select_dtypes(include=np.number).columns.tolist())
```

```
Numerical Features in Dataset 1
['OBS', 'DURATION', 'NEW_CAR', 'USED_CAR', 'FURNITURE', 'RADIO_TV', 'EDUCATIO
N', 'RETRAINING', 'AMOUNT', 'INSTALL_RATE', 'CO_APPLICANT', 'GUARANTOR', 'REA
L_ESTATE', 'PROP_UNKN_NONE', 'AGE', 'OTHER_INSTALL', 'RENT', 'OWN_RES', 'NUM_CR
EDITS', 'NUM_DEPENDENTS', 'TELEPHONE', 'FOREIGN', 'RESPONSE']
```

In [61]:
```
# DATASET 2

print("\nNumerical Features in Dataset 2")
print(df2.select_dtypes(include=np.number).columns.tolist())
```

```
Numerical Features in Dataset 2
['OBS', 'CHK_ACCT', 'HISTORY', 'SAV_ACCT', 'EMPLOYMENT', 'PRESENT_RESIDENT', 'J
OB']
```

In [62]:
```
### TASK 4:- Use info() to check missing values.

## DATASET 1

print("\nDataset 1 Information")
df1.info()
```

```
Dataset 1 Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 23 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   OBS             1000 non-null   int64
 1   DURATION        998 non-null    float64
 2   NEW_CAR         999 non-null    float64
 3   USED_CAR        999 non-null    float64
 4   FURNITURE       999 non-null    float64
 5   RADIO_TV        997 non-null    float64
 6   EDUCATION       996 non-null    float64
 7   RETRAINING      996 non-null    float64
 8   AMOUNT          990 non-null    float64
 9   INSTALL_RATE    993 non-null    float64
 10  CO_APPLICANT    992 non-null    float64
 11  GUARANTOR       993 non-null    float64
 12  REAL_ESTATE     993 non-null    float64
 13  PROP_UNKN_NONE  993 non-null    float64
 14  AGE             990 non-null    float64
 15  OTHER_INSTALL   991 non-null    float64
 16  RENT            992 non-null    float64
 17  OWN_RES         993 non-null    float64
 18  NUM_CREDITS     994 non-null    float64
 19  NUM_DEPENDENTS  993 non-null    float64
 20  TELEPHONE       995 non-null    float64
 21  FOREIGN         994 non-null    float64
 22  RESPONSE        997 non-null    float64
dtypes: float64(22), int64(1)
memory usage: 179.8 KB
```

In [63]:
```
## DATASET 2

print("\nDataset 2 Information")
df2.info()
```

```
Dataset 2 Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   OBS               1000 non-null   int64
 1   CHK_ACCT          1000 non-null   int64
 2   HISTORY           1000 non-null   int64
 3   SAV_ACCT          1000 non-null   int64
 4   EMPLOYMENT        1000 non-null   int64
 5   PRESENT_RESIDENT  1000 non-null   int64
 6   JOB               1000 non-null   int64
dtypes: int64(7)
memory usage: 54.8 KB
```

In [64]:
```
#### Data Exploration

### TASK 1:- Find the average credit amount (AMOUNT).

print("Average Credit Amount =",
      df1["AMOUNT"].mean())
```

```
Average Credit Amount = 3273.168686868687
```

In [65]:
```
### TASK 2:- Find the maximum and minimum AGE.

print("Maximum Age =", df1["AGE"].max())

print("Minimum Age =", df1["AGE"].min())
```

```
Maximum Age = 75.0
Minimum Age = 19.0
```

In [66]:
```
### TASK 3:- Count customers with RESPONSE = 1

response_count = (df1["RESPONSE"] == 1).sum()
print("Customers with RESPONSE = 1 =", response_count)
```

```
Customers with RESPONSE = 1 = 698
```

In [67]:
```
### TASK 4:- Find the distribution of INSTALL_RATE

print("Distribution of INSTALL_RATE")
print(df1["INSTALL_RATE"].value_counts())
```

```
Distribution of INSTALL_RATE
INSTALL_RATE
4.0    473
2.0    230
3.0    155
1.0    135
Name: count, dtype: int64
```

In [68]:
```
#### Data Manipulation

### TASK 1:- Create a new column: AMOUNT_PER_DURATION

df1["AMOUNT_PER_DURATION"] = df1["AMOUNT"] / df1["DURATION"]
print("AMOUNT_PER_DURATION")
display(df1[["AMOUNT", "DURATION", "AMOUNT_PER_DURATION"]].head())
```

```
AMOUNT_PER_DURATION
```

	AMOUNT	DURATION	AMOUNT_PER_DURATION
0	1169.0	6.0	194.833333
1	5951.0	48.0	123.979167
2	2096.0	12.0	174.666667
3	7882.0	42.0	187.666667
4	4870.0	24.0	202.916667

In [69]:
```
### TASK 2:- Filter customers with AGE > 40 and AMOUNT > 5000

filtered_data = df1[ (df1["AGE"] > 40) & (df1["AMOUNT"] > 5000) ]
print("Filtered Customers")
display(filtered_data.head())

print("Total Matching Customers:", len(filtered_data))
```

```
Filtered Customers
```

	OBS	DURATION	NEW_CAR	USED_CAR	FURNITURE	RADIO_TV	EDUCATION	R
3	4	42.0	0.0	0.0	1.0	0.0	0.0	
18	19	24.0	0.0	1.0	0.0	0.0	0.0	
29	30	60.0	0.0	0.0	0.0	0.0	0.0	
42	43	18.0	0.0	0.0	0.0	0.0	0.0	
44	45	48.0	0.0	1.0	0.0	0.0	0.0	

5 rows × 24 columns

```
Total Matching Customers: 60
```

In [70]:
```
### TASK 3:- Sort data based on AMOUNT (descending)
```

```
sorted_data = df1.sort_values( by="AMOUNT", ascending=False )
print("Top 10 Customers by Amount")
display(sorted_data.head(10))
```

Top 10 Customers by Amount

	OBS	DURATION	NEW_CAR	USED_CAR	FURNITURE	RADIO_TV	EDUCATION
915	916	48.0	0.0	0.0	0.0	0.0	0.0
95	96	54.0	0.0	0.0	0.0	0.0	0.0
818	819	36.0	0.0	0.0	0.0	0.0	0.0
887	888	48.0	0.0	0.0	0.0	0.0	0.0
637	638	60.0	0.0	0.0	0.0	1.0	0.0
917	918	6.0	1.0	0.0	0.0	0.0	0.0
374	375	60.0	0.0	0.0	0.0	0.0	0.0
236	237	6.0	1.0	0.0	0.0	0.0	0.0
63	64	48.0	0.0	0.0	0.0	0.0	0.0
378	379	36.0	1.0	0.0	0.0	0.0	0.0

10 rows × 24 columns

In [71]:
```
#### Grouping & Analysis

### TASK 1:- Group by RESPONSE and find mean AMOUNT

group_amount = df1.groupby("RESPONSE")["AMOUNT"].mean()
print("Mean Amount by RESPONSE")
print(group_amount)
```

```
Mean Amount by RESPONSE
RESPONSE
0.0    3946.831081
1.0    2986.940837
Name: AMOUNT, dtype: float64
```

In [72]:
```
### TASK 2:- Find average AGE for each RESPONSE

group_age = df1.groupby("RESPONSE")["AGE"].mean()
print("Average Age by RESPONSE")
print(group_age)
```

```
Average Age by RESPONSE
RESPONSE
0.0    33.787162
1.0    36.207792
Name: AGE, dtype: float64
```

In [73]:
```
#### Merging & Advanced

### TASK 1:- Merge both datasets using OBS column.

merged_df = pd.merge( df1, df2, on="OBS" )
print("Merged Dataset Shape")
print(merged_df.shape)
display(merged_df.head())
```

```
Merged Dataset Shape
(1000, 30)
```

	OBS	DURATION	NEW_CAR	USED_CAR	FURNITURE	RADIO_TV	EDUCATION	RE
0	1	6.0	0.0	0.0	0.0	1.0	0.0	
1	2	48.0	0.0	0.0	0.0	1.0	0.0	
2	3	12.0	0.0	0.0	0.0	0.0	1.0	
3	4	42.0	0.0	0.0	1.0	0.0	0.0	
4	5	24.0	1.0	0.0	0.0	0.0	0.0	

5 rows × 30 columns

In [74]:
```
### TASK 2:- Find correlation between numerical variables.

correlation_matrix = merged_df.corr( numeric_only=True )

print("Correlation Matrix")
display(correlation_matrix)

print("Correlation with RESPONSE")
display( correlation_matrix["RESPONSE"]
        .sort_values(ascending=False))
```

```
Correlation Matrix
```

	OBS	DURATION	NEW_CAR	USED_CAR	FURNITURE
OBS	1.000000	0.030324	0.058738	0.008841	-0.007529
DURATION	0.030324	1.000000	-0.111228	0.145838	-0.059562
NEW_CAR	0.058738	-0.111228	1.000000	-0.187518	-0.259281
USED_CAR	0.008841	0.145838	-0.187518	1.000000	-0.158949
FURNITURE	-0.007529	-0.059562	-0.259281	-0.158949	1.000000
RADIO_TV	-0.017823	-0.043652	-0.345212	-0.211587	-0.291601
EDUCATION	-0.024465	0.003021	-0.127044	-0.078079	-0.107611
RETRAINING	-0.017114	0.163451	-0.181519	-0.111558	-0.153752
AMOUNT	0.007981	0.624527	-0.038799	0.255915	-0.037997
INSTALL_RATE	0.012601	0.073696	-0.048154	-0.089643	-0.061209
CO_APPLICANT	0.017953	0.028763	0.004420	-0.053618	0.060602
GUARANTOR	-0.004431	-0.040825	-0.012816	-0.034865	-0.027914
REAL_ESTATE	-0.038536	-0.248557	0.043835	-0.130937	-0.055155
PROP_UNKN_NONE	-0.015777	0.213294	0.025373	0.129952	-0.070637
AGE	-0.000972	-0.036725	0.070218	0.059165	-0.129738
OTHER_INSTALL	-0.002021	0.063064	-0.032474	-0.008875	-0.002799
RENT	0.025078	-0.060467	-0.008678	0.032944	0.109966
OWN_RES	-0.016263	-0.075505	-0.008677	-0.138206	-0.045252
NUM_CREDITS	0.029534	-0.013449	0.027690	-0.002629	-0.068753
NUM_DEPENDENTS	0.024881	-0.025855	0.104698	0.055555	-0.087126
TELEPHONE	0.000594	0.165170	-0.041186	0.134376	-0.050794
FOREIGN	-0.018974	-0.139411	0.156039	-0.031466	-0.009663
RESPONSE	-0.037210	-0.211943	-0.095439	0.099893	-0.022867
AMOUNT_PER_DURATION	-0.013978	-0.125942	0.105883	0.109085	0.005086
CHK_ACCT	0.005852	-0.070747	-0.068987	0.064733	-0.100719
HISTORY	-0.011691	-0.075708	0.042213	0.038933	-0.024526
SAV_ACCT	0.003730	0.047529	-0.002736	0.112677	-0.080954
EMPLOYMENT	-0.020078	0.055196	-0.020956	0.039538	-0.081938
PRESENT_RESIDENT	0.023697	0.034497	0.020442	0.107677	-0.009189
JOB	-0.027345	0.213689	-0.089576	0.180444	0.016421

30 rows × 30 columns

Correlation with RESPONSE

	RESPONSE
RESPONSE	1.000000
CHK_ACCT	0.349708
HISTORY	0.229500
SAV_ACCT	0.179208
OWN_RES	0.131695
EMPLOYMENT	0.118237
REAL_ESTATE	0.116535
RADIO_TV	0.108934
USED_CAR	0.099893
AGE	0.097908
FOREIGN	0.081913
GUARANTOR	0.054994
NUM_CREDITS	0.046197
TELEPHONE	0.036227
NUM_DEPENDENTS	0.002179
PRESENT_RESIDENT	-0.002747
AMOUNT_PER_DURATION	-0.019965
FURNITURE	-0.022867
JOB	-0.031783
OBS	-0.037210
RETRAINING	-0.037365
CO_APPLICANT	-0.063180
EDUCATION	-0.071132
INSTALL_RATE	-0.073559
RENT	-0.092021
NEW_CAR	-0.095439
OTHER_INSTALL	-0.110546
PROP_UNKN_NONE	-0.123057
AMOUNT	-0.155549
DURATION	-0.211943

dtype: float64

In [75]:
```
#### NumPy Tasks

### Create a 3×4 NumPy array and print shape, size, dtype

array1 = np.array([ [1,2,3,4], [5,6,7,8], [9,10,11,12] ])
print(array1)

print("Shape:", array1.shape)
print("Size:", array1.size)
print("Datatype:", array1.dtype)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape: (3, 4)
Size: 12
Datatype: int64
```

In [80]:
```
### Task 2:- Create arrays using zeros(), ones(), and full()

print("Zeros Array")

print(np.zeros((3,3)))

print("\nOnes Array")

print(np.ones((3,3)))

print("\nFull Array")

print(np.full((3,3), 5))
```

```
Zeros Array
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

Ones Array
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]

Full Array
[[5 5 5]
 [5 5 5]
 [5 5 5]]
```

In [81]:
```
### Task 3:- Generate numbers using arange() and linspace().

print("Using arange")

print(np.arange(0,21,2))
```

```
print("\nUsing linspace")

print(np.linspace(0,100,5))
```

```
Using arange
[ 0  2  4  6  8 10 12 14 16 18 20]

Using linspace
[  0.  25.  50.  75. 100.]
```

In [82]:
```
### Task 4:- Create a random array and fix randomness using seed.

np.random.seed(42)

random_array = np.random.randint(
    1,
    100,
    (3,3)
)

print(random_array)
```

```
[[52 93 15]
 [72 61 21]
 [83 87 75]]
```

In [83]:
```
### Task 5:- Perform indexing (first, last, and specific element)

arr = np.array([10,20,30,40,50])

print("First Element:", arr[0])

print("Last Element:", arr[-1])

print("Specific Element:", arr[2])
```

```
First Element: 10
Last Element: 50
Specific Element: 30
```

In [84]:
```
### Task 6:- Slice a 2D array to extract a submatrix

matrix = np.array([
    [1,2,3,4],
    [5,6,7,8],
    [9,10,11,12]
])

submatrix = matrix[0:2,1:3]

print(submatrix)
```

```
[[2 3]
 [6 7]]
```

In [85]:
```
### Task 7:- Reshape a 1D array into 2D array

one_d = np.arange(1,13)

two_d = one_d.reshape(3,4)

print(two_d)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

In [86]:
```
### Task 8:- Perform transpose of a matrix.

matrix = np.array([
    [1,2,3],
    [4,5,6]
])

print("Original Matrix")

print(matrix)

print("\nTranspose")

print(matrix.T)
```

```
Original Matrix
[[1 2 3]
 [4 5 6]]

Transpose
[[1 4]
 [2 5]
 [3 6]]
```

In [87]:
```
### Task 9:- Demonstrate difference between view and copy.

arr = np.array([1,2,3,4,5])

view_arr = arr.view()

copy_arr = arr.copy()

arr[0] = 100

print("Original:", arr)

print("View:", view_arr)

print("Copy:", copy_arr)
```

```
Original: [100   2   3   4   5]
View: [100   2   3   4   5]
Copy: [1 2 3 4 5]
```

In [88]:
```
### Task 10:- Reverse an array using slicing

arr = np.array([1,2,3,4,5])

reverse_arr = arr[::-1]

print(reverse_arr)
```

```
[5 4 3 2 1]
```